

RFC 8520 · RASPBERRY PI 3B+ · IPTABLES · MOSQUITTO · PYTHON

MUD-Based IoT Security Gateway

Project guide · how to run it · viva and interview questions

TEAM

Goutam · Joshua Lewis · Dhanush S

GUIDE

Dr. Nagaraja B G, Professor and Associate Dean, ECE

INSTITUTION

Vidyavardhaka College of Engineering, Mysuru

VERSION

8 October 2026 · all six research gaps implemented and measured

100%

enforcement accuracy (87/87) MQTT topic attacks stopped

20→100%

166 ms

policy hot-reload

36/36

same-WiFi attacks blocked

Contents

Contents	2
1. The project on one page	3
2. Background explained simply	4
3. Our architecture	5
4. How it works, step by step	6
5. What is new: the six research gaps we closed	9
6. Testing method and results	12
7. How to start the project and show every output	13
8. A 5-minute presentation flow	16
9. Limitations and future work	17
10. Viva and interview questions with answers	17

How to use this guide

Read sections 1 to 4 to **understand** the project. Use section 7 to **run and demonstrate** it. Section 10 is the **viva and interview** preparation: read each answer out loud until you can say it in your own words.

1. The project on one page

Problem. Smart devices such as cameras, sensors and smart locks are easy to hack and hard to patch. Once hacked, they attack other devices, join botnets such as Mirai, or leak private data. A normal home router does not know what each device is supposed to do, so it cannot tell normal traffic from attack traffic.

Idea (RFC 8520, MUD). The **manufacturer** publishes a small, digitally signed file, the **Manufacturer Usage Description**, that lists exactly what the device needs on the network, for example 'talk to my cloud server on port 8886'. The network reads this file and blocks everything else.

What we built. A Raspberry Pi 3B+ that works as the WiFi gateway for real IoT devices and enforces their MUD files:

- A real **CP Plus CP-E35Q WiFi camera** and a real **DHT11 sensor** are protected by the Pi.
- The Pi **downloads each MUD file, checks its RSA-2048 signature** and turns it into Linux firewall (iptables) rules.
- Everything the device is not allowed to do is **logged and dropped** (default deny).
- On top of plain RFC 8520 we added: **same-WiFi (east-west) protection, MQTT topic-level control, automatic signed policy updates** with an atomic rule swap, **start-up at boot**, and **measured results** for all of it.

Key results (measured on the real Pi)

What we measured	Result	How
Enforcement accuracy	100% (87/87 trials)	29 test connections × 3 trials, baseline vs enforced
Same-WiFi attacks on the camera	36/36 blocked	Laptop on the same WiFi attacking 12 camera ports × 3
MQTT topic attacks	20% → 100% accuracy	10 attack/legit cases × 3, port-level vs topic ACL
Policy hot-reload	166 ms apply (median), 1.12 s end-to-end	10 signed updates
Signature check (RSA-2048, PSS, SHA-256)	about 1.3-1.5 ms	watchdog timing log
Hardware	Off-the-shelf parts only	Itemised bill of materials in docs/hardware_bom.csv
Recovery after reboot	Protected about 10 s after services start	systemd services, journal timestamps

The one sentence to remember for the panel

MUD is already an IETF standard, so 'we implemented RFC 8520' alone is not new. Our contribution is **closing six gaps** found in the literature survey (section 5) on real hardware, with numbers to prove each one.

2. Background explained simply

2.1 What is MUD? An analogy

Think of a **hostel visitor register**. The warden (our Raspberry Pi) does not know every visitor. So each student's parents (the manufacturer) send a signed letter (the MUD file) saying: 'my son may go to the college library and the mess, nowhere else'. The warden checks the parents' signature, then lets the student go only to those places. If someone forges the letter, the signature check fails and the old rules stay.

2.2 The RFC 8520 words you must know

Term	Simple meaning	In our project
MUD file	JSON file listing what a device may talk to	mud_files/ip_camera.json, temperature_sensor.json
MUD URL	Web address of the MUD file. A real device announces it (DHCP option 161, LLDP, or its X.509 certificate)	Configured on the gateway, because our camera does not announce one
MUD file server	The manufacturer's web server that hosts the file	Flask app on the Pi, port 5000 (mud_file_server/server.py)
MUD manager	Software that fetches, checks and applies MUD files	gateway_enforcer.py + watchdog_monitor.py
ACL / ACE	Access Control List / Entry: one allow or deny line (format from RFC 8519)	e.g. 'TCP to m6-cube.cppluscloud.com port 8886: accept'
from-device / to-device policy	Rules for traffic leaving / entering the device	chains MUD_CAM_OUT / MUD_CAM_IN
Default deny	Anything not listed is blocked	Last rules of each chain: LOG + DROP
Signature	Proof the file came from the manufacturer and was not changed	RSA-2048 with PSS padding and SHA-256

2.3 Networking basics used in this project

Word	Plain explanation
IP address	The 'house address' of a device, e.g. camera 10.42.0.17, Pi 10.42.0.1.
Port	The 'room number' inside the house for a service: 53 = DNS, 443 = HTTPS, 1883 = MQTT, 23 = Telnet.
TCP / UDP	Two ways to send data. TCP sets up a connection and confirms delivery; UDP just sends (fast, no confirmation).
DNS	Phone book of the internet: turns m6-cube.cppluscloud.com into an IP address.
DHCP	The service that hands out IP addresses when a device joins WiFi.
NAT	The Pi rewrites the camera's private address into its own when talking to the internet (like a reception desk forwarding calls).
MAC address / ARP	MAC is the hardware address of a network card. ARP is the question 'who has IP 10.42.0.17? tell me your MAC'.
TTL	A counter in every packet that drops by 1 at every router. 64 at the camera means 'no router in between'.
iptables	Linux's built-in firewall. Rules sit in chains. INPUT = to the Pi itself, OUTPUT = from the Pi, FORWARD = passing through the Pi.

Word	Plain explanation
conntrack	Linux remembers open connections. 'ESTABLISHED' = a reply to a connection that was already allowed.
MQTT	Light messaging for IoT. Devices publish to a topic (e.g. sensors/temperature) on a broker (Mosquitto); others subscribe. '#' = every topic.
systemd	Linux's service manager. It starts our programs automatically at boot and restarts them if they crash.

3. Our architecture

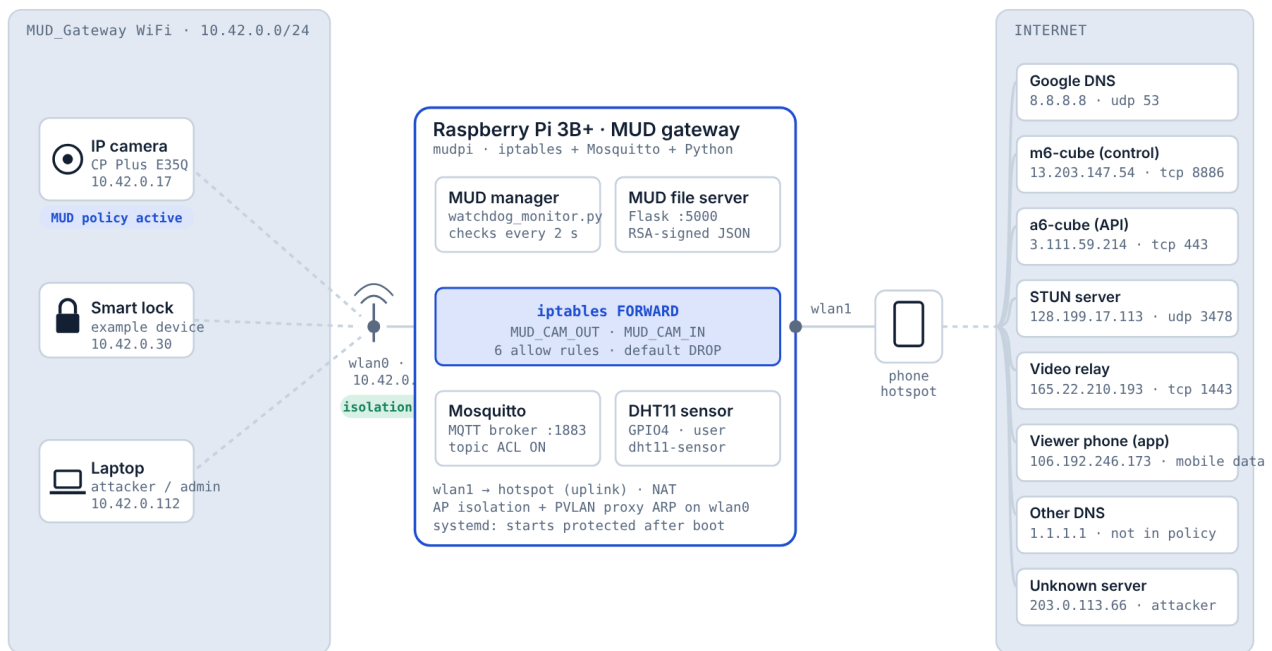


Figure 1. The real setup (drawn by our simulator). Every address is from the live network.

3.1 Hardware

Part	Job
Raspberry Pi 3B+ (1 GB)	The gateway: runs the MUD manager, firewall, MQTT broker
microSD card 16 GB	Raspberry Pi OS Lite (32-bit) + project
TP-Link TL-WN725N USB WiFi (RTL8188EUS)	wlan1: connects the Pi to the phone hotspot (internet)
Built-in WiFi (BCM43455)	wlan0: the access point MUD_Gateway (10.42.0.1/24) for the devices
5V/2A charger	Power. A laptop USB port caused undervoltage, so we use a charger
CP Plus CP-E35Q camera	Real IoT device under test, IP 10.42.0.17
DHT11 sensor on GPIO4	Real sensor; publishes over MQTT to the Pi
USB card reader	One-time, to flash the OS

Why two WiFi radios? The dongle supports only client and monitor mode, not access-point mode. So the built-in radio must be the AP (wlan0) and the dongle connects upstream (wlan1). We proved this with `iw phy phy1 info`.

3.2 Software components

File / service	What it does
mud_file_server/server.py (mud-server.service)	Serves MUD files and signs every response with the manufacturer's RSA private key (header X-MUD-Signature). Debug mode off.
mud_manager/gateway_enforcer.py	One-shot: fetch → verify → parse → install chains MUD_<NAME>_OUT/IN on FORWARD. Also --remove.
mud_manager/watchdog_monitor.py (mud-watchdog-cam.service)	Checks the MUD file every 2 s. If it changed: verify signature, parse, swap all rules in one atomic transaction, log the reload.
mud_manager/mqtt_acl_manager.py	Reads the MUD extension mqtt-topic-acl and writes Mosquitto users, passwords and topic permissions.
mud_files/ip_camera.json	Camera policy written from its measured traffic (hybrid: DNS names + port rules).
mud_files/temperature_sensor.json	Sensor policy with the mqtt-topic-acl extension.
real_sensor.py	Reads the DHT11 and publishes to sensors/temperature using its MUD-issued login.
tests/attack_test.py	Runs on the laptop: 29 allowed/forbidden connections, baseline vs enforced, accuracy %.
mud_manager/reload_latency_test.py	Publishes 10 signed policy changes and times how fast the kernel rules change.
mud_manager/mqtt_acl_test.py	Hacked-sensor test: 10 MQTT cases × 3 trials.
/etc/sysctl.d/99-mud-gateway.conf	Kernel settings for proxy ARP (east-west protection), loaded at every boot.
docs/MUD_Gateway_Simulator_offline.html	Interactive simulation for presentations.

4. How it works, step by step



Step 1. The device joins

The camera connects to the WiFi network **MUD_Gateway**. NetworkManager's DHCP gives it 10.42.0.17. Because the Pi is the only way to the internet, every packet the camera sends must pass through the Pi's **FORWARD** chain.

Step 2. Fetch the MUD file

The MUD manager requests `http://127.0.0.1:5000/mud/ip_camera.json`. The server returns the JSON and a header X-MUD-Signature made with the private key.

Step 3. Verify the signature

gateway_enforcer.py / watchdog_monitor.py

```
public_key.verify(
    bytes.fromhex(sig_hex), raw_bytes,
    padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256())          # raises InvalidSignature -> file REJECTED, no rules changed
```

Only the holder of the **private** key can make a valid signature; anyone can check it with the **public** key. We first used HMAC (a shared secret) and upgraded to RSA because with HMAC the gateway itself could forge files.

Step 4. Parse: MUD JSON → firewall rules

One ACE from ip_camera.json

```
{ "name": "cloud-control-8886",
  "matches": { "ipv4": { "ietf-acldns:dst-dnsname": "m6-cube.cppluscloud.com" },
               "tcp": { "destination-port": { "operator": "eq", "port": 8886 } } },
  "actions": { "forwarding": "accept" } }
```

The parser accepts both the official RFC 8519 key names and older short names, handles TCP and UDP, keeps the direction (from-device / to-device), and resolves DNS names into IP addresses.

Step 5. Install: the real chains on the Pi

MUD_CAM_OUT		from camera · first match wins
0	ACCEPT	ctstate RELATED,ESTABLISHED replies to allowed flows
0	ACCEPT	udp dpt:53 → 8.8.8.8 cam-from-dns
0	ACCEPT	tcp dpt:8886 → 13.203.132.148 m6-cube.cppluscloud.com
0	ACCEPT	tcp dpt:8886 → 13.203.147.54 m6-cube.cppluscloud.com
0	ACCEPT	tcp dpt:443 → 3.111.59.214 a6-cube.cppluscloud.com
0	ACCEPT	udp dpt:3478 → any cam-from-stun
0	ACCEPT	tcp dpt:1443 → any cam-from-video-relay
0	LOG	prefix "MUD_CAM_FROM_DROP: " cam-from-deny-all
0	DROP	everything else default deny
MUD_CAM_IN		to camera
0	ACCEPT	ctstate RELATED,ESTABLISHED replies to allowed flows
0	LOG	prefix "MUD_CAM_TO_DROP: " cam-to-deny-all
0	DROP	everything else default deny

Figure 2. MUD_CAM_OUT and MUD_CAM_IN as installed. The first matching rule decides.

sudo iptables -L FORWARD -n

```
Chain FORWARD (policy ACCEPT)
MUD_CAM_IN  all  --  0.0.0.0/0      10.42.0.17
MUD_CAM_OUT all  --  10.42.0.17     0.0.0.0/0
```

Step 6. Enforce

Allowed flows pass and are counted. Anything else hits the LOG rule (rate-limited to 10 per minute) and then DROP. A real line from the kernel log:

```
MUD_CAM_FROM_DROP: IN=wlan0 OUT=wlan1 SRC=10.42.0.17 DST=106.192.246.173
PROTO=UDP SPT=39986 DPT=55952
```

The camera's policy: written from its real traffic

We captured the camera's traffic on the Pi (tcpdump for DNS, conntrack for connections) and wrote the MUD file from what we saw:

Flow	Destination	Rule type	Why
DNS (UDP 53)	8.8.8.8 (hard-coded in the camera)	fixed IP	name lookups
Control (TCP 8886)	m6-cube.cppluscloud.com (AWS, Mumbai)	DNS name	permanent cloud connection
API (TCP 443)	a6-cube.cppluscloud.com	DNS name	short HTTPS calls
STUN (UDP 3478)	any	port only	live view: find public address
Video relay (TCP 1443)	any	port only	live video through relay servers
Everything else	-	LOG + DROP	default deny

A finding worth mentioning in the viva

RFC 8520 lets a MUD file name destinations by DNS name. But the camera's STUN and relay servers are picked at run time and **never appeared in DNS**, so a pure DNS-name policy would break live view. We used a **hybrid policy**: DNS names for fixed cloud services, port rules for dynamic video servers. This is a real limitation of RFC 8520 that we found and documented.

5. What is new: the six research gaps we closed

The guide's feedback was that MUD already exists, so the project must show what is **new**. Our literature survey (28 reviewed contributions) found six gaps. Each one is now closed with working code and a measurement.

Gap	What was missing in the papers	What we built	Evidence
5.1 Simulation vs hardware	Most work is simulated (Mininet etc.)	Real Pi, real camera, real DHT11, real attacks	All tests on live hardware
5.2 Enterprise dependence	Needs managed switches / VLANs / SDN	East-west isolation with AP isolation + PVLAN proxy ARP on a Raspberry Pi	36/36 same-WiFi attacks blocked
5.3 MQTT topic enforcement	Only IP/port rules; MQTT topics invisible	MUD extension mqtt-topic-acl → Mosquitto ACL + per-device login	20% → 100% (30/30)
5.4 Dynamic policy updates	Policies installed once, never updated safely	Watchdog: detect change, verify, atomic swap, log	166 ms apply, 1.12 s end-to-end
5.5 Explainability / audit	No record of why traffic was blocked	Kernel drop logs, reload log CSV, test CSVs	logs/policy_reload_log.csv, tests/*.csv
5.6 Cost & reproducibility	No cost, no numbers, no code	BOM, accuracy and latency metrics, full repo	Itemised BOM; 100% accuracy

5.2 East-west protection (same-WiFi attacks)

The problem we found. A laptop on the same WiFi pinged the camera and got replies with **TTL=64**, while the MUD counters stayed at 0. The WiFi chip was passing frames directly between the two devices, so the packets never entered Linux and iptables never saw them.

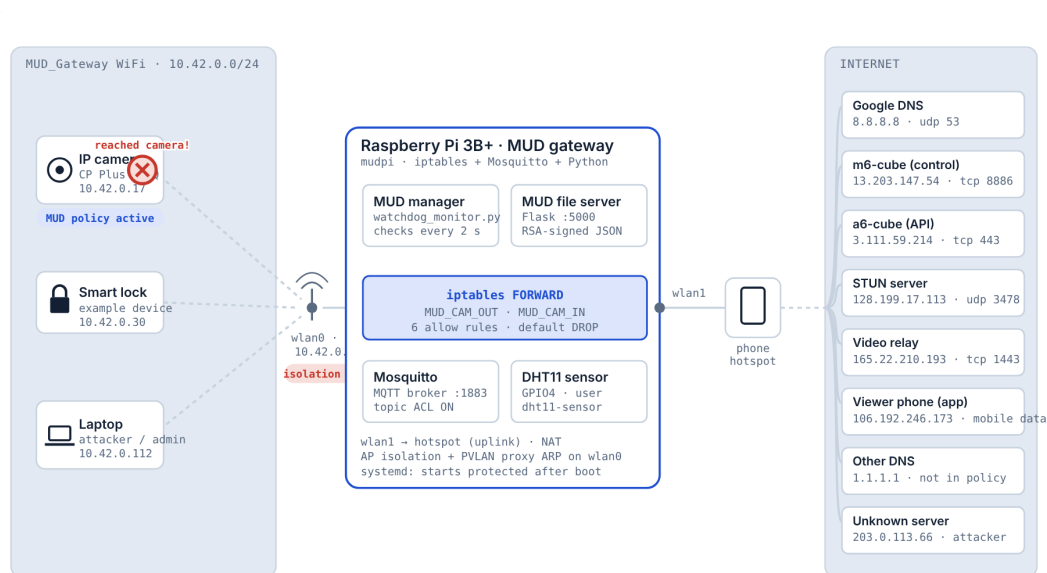


Figure 3. Without isolation the attack travels laptop → radio → camera and never reaches the firewall.

The fix, in two parts:

- **AP isolation** (802-11-wireless.ap-isolation yes): the radio refuses to pass frames between clients.
- **Proxy ARP for private VLANs**: the Pi answers every ARP question with its own MAC address, so client-to-client packets are sent to the Pi, go through FORWARD (they 'hairpin' in and out of wlan0) and meet the camera's MUD_CAM_IN chain.

`/etc/sysctl.d/99-mud-gateway.conf`

```
net.ipv4.conf.wlan0.proxy_arp = 1           # Pi answers ARP for others
net.ipv4.conf.wlan0.proxy_arp_pvlan = 1    # ...even on the same interface (RFC 3069 style)
net.ipv4.conf.all.send_redirects = 0       # don't tell the laptop "go direct"
net.ipv4.conf.wlan0.send_redirects = 0
```

Proof from the kernel log: MUD_CAM_TO_DROP: IN=wlan0 OUT=wlan0 SRC=10.42.0.112 DST=10.42.0.17 TTL=127. TTL 127 means the packet passed through one router, the Pi. Enterprises do this with Private-VLAN switches; we did it with one WiFi setting and three kernel settings.

5.3 MQTT topic-level enforcement

The firewall can only say 'TCP to port 1883 is allowed'. Inside that connection a hacked sensor could publish a fake home/door/unlock command or subscribe to # and spy on everything.

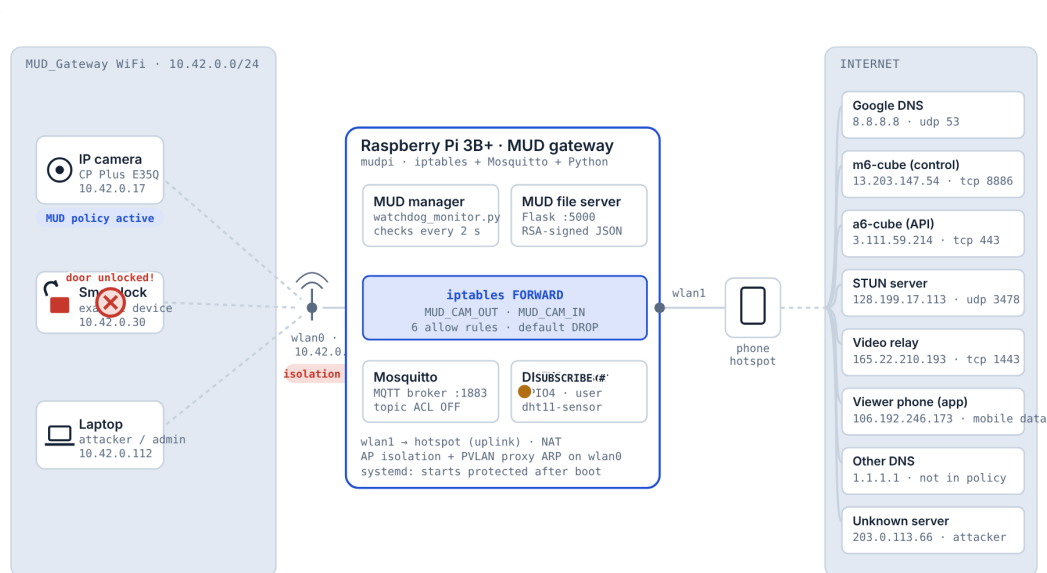


Figure 4. Port-level MUD only: the hacked sensor's fake command reaches the smart lock (example device).

We added an extension to the signed MUD file (RFC 8520 allows extensions through its extensions list):

mud_files/temperature_sensor.json

```
"extensions": ["mqtt-topic-acl"],
"mqtt-topic-acl:policy": {
  "username": "dht11-sensor",
  "publish": ["sensors/temperature"],
  "subscribe": ["sensors/temperature/config"] }
```

mqtt_acl_manager.py verifies the file's signature and generates Mosquitto's configuration: no anonymous clients, one login per device (password stored hashed), and a topic ACL. Generated passwords live in `device_creds/`, which is kept out of git.

/etc/mosquitto/mud_acl (generated)

```
user dht11-sensor
topic write sensors/temperature
topic read sensors/temperature/config
```

5.4 Dynamic policy updates with an atomic swap

- Every 2 s the watchdog downloads the MUD file and compares its **SHA-256 fingerprint** with the last one.
- If it changed, the **signature is checked first**. A forged file is rejected and the old rules stay in force.
- The complete new chains are given to the kernel in **one iptables-restore transaction**.
- Every reload is written to `logs/policy_reload_log.csv`.

Why atomic? The old method empties a chain and adds rules one by one (0.2–0.7 s on the Pi). During that moment the chain is empty, an empty chain returns packets to FORWARD, and FORWARD's default policy is ACCEPT: the gateway would **fail open**. With one transaction, packets always see either the whole old policy or the whole new one.

6. Testing method and results

6.1 Enforcement accuracy (attack_test.py)

The laptop plays two roles: an attacker on the same WiFi (12 tests against the camera) and a **hacked camera**. For the second role the Pi gives the laptop exactly the camera's policy (enforcer name ATK). 5 tests must pass and 12 must be blocked.

- **Baseline first.** Run with no MUD rules. Only tests that get through in the baseline are counted, so a timeout caused by a dead server can never be mistaken for MUD blocking.
- **Off-path check.** Before and after every trial the script checks the laptop is still sending through 10.42.0.x. (Windows once silently switched WiFi; we fixed it by setting all other networks to manual.)
- **Canary check.** A forbidden connection (1.1.1.1:443) is tried before and after the run. If it gets through, the run is marked INVALID.
- Each test runs 3 times; results are saved as CSV.

Result	Value
Tests counted / excluded	29 / 0
Forbidden trials blocked (TP)	72 / 72
Allowed trials passed (TN)	15 / 15
Enforcement accuracy	100% (87/87)
Key cases	B07: 443 to 1.1.1.1 blocked (right port, wrong destination); B08: 8886 to a non-cloud server blocked

6.2 Hot-reload latency (reload_latency_test.py)

Stage	Time
Detection (waiting for the next check)	0-2 s, about 1 s on average (set by --interval 2)
Fetch (HTTP)	about 43 ms
Signature verify	about 1.3 ms
Parse	about 0.9 ms
Atomic install	about 110-140 ms
Apply total	median 166 ms (156-186), 10 trials
End-to-end	median 1.12 s (0.64-1.16)

6.3 MQTT topic ACL (mqtt_acl_test.py)

Case	Port-level only	With topic ACL
M01-M02 sensor's own publish / config (legitimate)	6/6 allowed	6/6 allowed
M03-M07 fake unlock, fake data, camera tampering, spying (#)	15/15 succeeded	0/15 succeeded
M08-M10 anonymous, wrong password, unknown device	9/9 accepted	0/9 accepted
Accuracy	20.0% (6/30)	100% (30/30)

6.4 Earlier Level-3 benchmarks (DHT11 / rogue device, still valid)

Metric	Value
ARP-based new-device detection (scapy)	~1,833 ms
RSA-2048 signature verification	~1.52 ms
MUD policy parsing	~1.17 ms
Rule installation (old one-by-one method)	~418 ms
Attack blocking, rogue_device.py	100% across 120 TCP trials

6.5 A real-world observation: P2P blocked, video kept working

When live view opened, the camera tried to send UDP directly to the viewer's phone on ports 55950–55952 (P2P 'hole punching'). That is not in the policy, so MUD dropped it (286 packets in one session). The app fell back to the allowed TCP 1443 relay and the video kept playing. This shows MUD can remove a risky behaviour without breaking the feature.

7. How to start the project and show every output

Read this first

Two terminals are used everywhere. **PI** = the SSH window whose prompt is `pi@mudpi`. **LAPTOP** = Windows PowerShell whose prompt is `PS C:\...` Never type Pi commands in PowerShell.

7.1 Power on and connect (2 minutes)

- Plug the Pi into the **5V/2A charger** (not the laptop USB port). Turn on the phone hotspot 'Galaxy A17 5G 2E03' first, because the Pi uses it for internet.
- Wait about 1.5 minutes. The camera joins MUD_Gateway by itself.
- On the laptop, join the WiFi **MUD_Gateway** (password in your notes). Then:

LAPTOP

```
ssh pi@10.42.0.1
```

7.2 Check that everything started by itself

PI

```
uptime
systemctl status mud-server mud-watchdog-cam --no-pager      # both: active (running)
sudo iptables -L FORWARD -n                                # MUD_CAM_IN and MUD_CAM_OUT present
journalctl -u mud-watchdog-cam -b --no-pager | head -5      # "INITIAL ... applied, 8 rule(s)"
systemctl is-active mosquitto                               # active
sysctl net.ipv4.conf.wlan0.proxy_arp_pvlan                  # = 1
nmcli -g 802-11-wireless.ap-isolation connection show MUD_Gateway # 1
ip neigh show dev wlan0                                     # camera 10.42.0.17 listed
```

7.3 Show the camera's live rules and counters

PI

```
sudo iptables -L MUD_CAM_OUT -n -v --line-numbers          # pkts column grows as the camera talks
sudo iptables -L MUD_CAM_IN -n -v
```

Open live view in the ezyKam+ app on a phone using **mobile data**. Run the command again: the counters on 8886, 3478 and 1443 increase.

7.4 Watch blocked packets live

PI

```
sudo journalctl -k -f | grep MUD_      # Ctrl+C to stop
```

Keep this window open during the other demos. Every blocked packet appears here with source, destination and port.

7.5 Demo: same-WiFi attack is blocked

LAPTOP

```
ping 10.42.0.17                # Request timed out
Test-NetConnection 10.42.0.17 -Port 554    # TcpTestSucceeded : False
```

The 7.4 window shows MUD_CAM_TO_DROP ... SRC=10.42.0.112 DST=10.42.0.17 TTL=127.

7.6 Demo: enforcement accuracy test (about 5 minutes)

The script is in F:\Major project\MUD_IoT\tests. The watchdog service manages the camera rules, so stop it for the baseline.

PI · step 1: no MUD rules

```
sudo systemctl stop mud-watchdog-cam
cd ~/mud_iot_project/mud_manager
sudo ../venv/bin/python3 gateway_enforcer.py --name CAM --ip 10.42.0.17 --remove
```

LAPTOP · step 2

```
cd "F:\Major project\MUD_IoT\tests"
python attack_test.py --label baseline      # must end with "Baseline OK"
```

PI · step 3: camera rules back + laptop gets the camera's policy for 60 s

```
sudo systemctl start mud-watchdog-cam
sudo ../venv/bin/python3 gateway_enforcer.py --name ATK --ip 10.42.0.112 \
  --url http://127.0.0.1:5000/mud/ip_camera.json && \
  echo ">>> ATK ACTIVE - run the laptop test NOW (60 s window)" && \
  sleep 60 && sudo iptables -L MUD_ATK_OUT -n -v && \
  sudo ../venv/bin/python3 gateway_enforcer.py --name ATK --ip 10.42.0.112 --remove
```

LAPTOP · step 4 (after the Pi prints 'Removed chains')

```
python attack_test.py --label enforced      # as soon as the Pi prints ATK ACTIVE
python attack_test.py --compare results_baseline.json results_enforced.json
```

While ATK is active the laptop's internet is cut (it is being treated as a camera), so keep this guide open on a phone. Expected: Enforcement accuracy 100.0% (87/87 trials).

7.7 Demo: signed policy hot-reload

PI · window 1

```
journalctl -u mud-watchdog-cam -f
```

PI · window 2

```
cd ~/mud_iot_project/mud_manager
sudo ../venv/bin/python3 reload_latency_test.py --name CAM \
  --file ../mud_files/ip_camera.json --trials 10
```

Window 1 prints a RELOAD line for every change. Window 2 prints the latency table. The original MUD file is restored automatically.

7.8 Demo: MQTT topic control

PI

```
cd ~/mud_iot_project/mud_manager
sudo ../venv/bin/python3 mqtt_acl_manager.py --disable          # port-level only
../venv/bin/python3 mqtt_acl_test.py --label baseline          # about 20%
sudo ../venv/bin/python3 mqtt_acl_manager.py \
  --url http://127.0.0.1:5000/mud/temperature_sensor.json      # topic ACL ON
../venv/bin/python3 mqtt_acl_test.py --label enforced          # 100% (30/30)
```

Always finish with the `--url` command so the ACL stays on.

7.9 Demo: the real DHT11 sensor

PI · window 1

```
cd ~/mud_iot_project
venv/bin/python3 real_sensor.py          # Ctrl+C to stop
```

PI · window 2: watch the readings arrive

```
sudo apt install -y mosquitto-clients # only once, if mosquitto_sub is missing
PW=$(python3 -c 'import json; print(json.load(open(
  "/home/pi/mud_iot_project/device_creds/mud-monitor.json"))["password"]))')
mosquitto_sub -h 127.0.0.1 -u mud-monitor -P "$PW" -t 'sensors/#' -v
```

7.10 Demo: the simulator

Open docs\MUD_Gateway_Simulator_offline.html in Chrome or Edge (works offline). It runs like slides: one idea per page, `→ / ←` keys or Next/Back to move, **▶ Play** (or Space) to animate, **Full screen** for the projector. Pages 8, 9 and 10 have a **Without our fix / With our fix** switch: show 'without' first, then 'with'. Page 11 lets the panel pick what the camera tries. Technical details for the examiner are under the button at the bottom of each page.

7.11 Manual start (if the services are stopped)

PI

```
sudo systemctl start mud-server mud-watchdog-cam          # normal way
# fully manual, in two SSH windows:
cd ~/mud_iot_project/mud_file_server && ../venv/bin/python3 server.py
cd ~/mud_iot_project/mud_manager
sudo ../venv/bin/python3 watchdog_monitor.py --name CAM --ip 10.42.0.17 \
  --url http://127.0.0.1:5000/mud/ip_camera.json --interval 2
```

7.12 Troubleshooting

Symptom	Cause	Fix
Connection refused on port 5000	MUD server not running	sudo systemctl restart mud-server; check journalctl -u mud-server

Symptom	Cause	Fix
'command not found: ../venv/bin/python3'	Wrong folder	cd ~/mud_iot_project/mud_manager first
'Sudo is disabled on this machine'	Pi command typed in PowerShell	Type it in the SSH window
Laptop has no internet	ATK rules still active	sudo ../venv/bin/python3 gateway_enforcer.py --name ATK --ip 10.42.0.112 --remove
Camera not on MUD_Gateway	Power / pairing	Use the charger; check ip neigh show dev wlan0; re-pair from a phone on MUD_Gateway
Undervoltage warnings	Laptop USB power	Use the 5V/2A charger
attack_test shows OFF-PATH	Windows switched WiFi	Set other WiFi profiles to manual (netsh wlan set profileparameter ... connectionmode=manual)
Mosquitto 'owner is not root' warning	File ownership preference	Harmless today; rules still enforced

7.13 Keeping laptop, GitHub and Pi in sync

LAPTOP or PI

```
# laptop folder: F:\Major project\MUD_IoT      Pi folder: ~/mud_iot_project
git pull                                     # before you start editing
git add <files>
git commit -m "what changed"
git push                                   # then git pull on the other machine
```

8. A 5-minute presentation flow

Time	Say	Show
0:00	IoT devices are easy to hack; routers don't know what each device should do.	Simulator, page 2
0:45	RFC 8520: the maker publishes a signed MUD file; the network enforces it.	Simulator, pages 3 and 5
1:30	Our Pi is the gateway for a real camera and DHT11.	Figure 1 / the hardware
2:00	Live rules and blocked packets.	Commands 7.3 and 7.4
2:45	What's new: east-west, MQTT topics, safe updates, boot, metrics.	Simulator, pages 8 to 10 (without / with fix)
4:00	Numbers: 100%, 36/36, 20→100%, 166 ms.	Simulator page 12 or section 6
4:40	Limitations and future work.	Section 9

9. Limitations and future work

- **MUD URL is configured, not announced.** The commercial camera does not send a MUD URL (DHCP option 161, LLDP or certificate). We acted as the manufacturer and wrote its MUD file from measured traffic.
- **Signature transport is simplified.** RFC 8520 uses a detached CMS (PKCS#7) signature file. We sign each response in an HTTP header with RSA-PSS. Same security idea, simpler format.
- **Identity is by IP address.** A device that steals the camera's IP/MAC would get the camera's rules. Future: WPA2/WPA3-Enterprise or per-device keys (802.1X).
- **Port-only rules for STUN/relay** are wider than ideal, because those servers never appear in DNS.
- **IPv4 only**, one gateway, polling every 2 s (not push notification).
- **MQTT without TLS.** Fine for a sensor on the Pi itself (localhost); a remote sensor would need TLS.
- **Development-grade server** (Flask built-in server, self-signed manufacturer key, no certificate chain).
- Future: auto-detect devices and pick their MUD file, a web dashboard of the logs, nftables, more device types.

10. Viva and interview questions with answers

Basics

Q1. Explain your project in two sentences.

We built a Raspberry Pi WiFi gateway that enforces RFC 8520 MUD policies for real IoT devices: it downloads each device's signed MUD file and turns it into firewall rules, so a device can only do what its manufacturer says it needs. On top of the standard we added same-WiFi protection, MQTT topic control, safe automatic policy updates and measured all of it.

Q2. What is MUD?

Manufacturer Usage Description, RFC 8520. A JSON file published by the device maker that lists the network access the device needs. The network enforces it and blocks everything else.

Q3. Why do IoT devices need this?

They are cheap, rarely updated and often have weak passwords. Botnets like Mirai hacked hundreds of thousands of cameras. A router that knows each device's normal behaviour can stop a hacked device from attacking others or leaking data.

Q4. How is MUD different from a normal firewall?

A normal firewall needs an admin to write rules for every device. With MUD the rules come from the manufacturer automatically, per device, and are signed, so no expert is needed at home.

Q5. What does 'default deny' mean?

Everything not explicitly allowed is blocked. Our chains end with LOG and DROP rules.

Q6. What are from-device and to-device policies?

From-device covers traffic the device starts (our chain MUD_CAM_OUT). To-device covers traffic sent to it (MUD_CAM_IN). For the camera, to-device is deny-all except replies.

Q7. What is RFC 8519?

The YANG data model for network access control lists. MUD files use its ACL and ACE format (matches + actions).

Q8. How does a real device tell the network its MUD URL?

Through a DHCP option (161 for IPv4), an LLDP extension, or an extension in its X.509 certificate. Our camera does none of these, so the gateway is configured with the URL. We state this as a limitation.

Hardware and network

Q9. Why a Raspberry Pi?

Low cost and widely available, runs full Linux with iptables, low power, and has built-in WiFi and GPIO pins for the DHT11 sensor.

Q10. Why two WiFi interfaces?

wlan0 (built-in) hosts the access point MUD_Gateway for the devices; wlan1 (USB dongle) connects to the phone hotspot for internet. The dongle cannot be an access point, which we checked with iw phy.

Q11. Why the FORWARD chain and not INPUT?

The camera's packets pass through the Pi on their way to the internet. In Linux, traffic passing through is handled by FORWARD. INPUT is only for traffic addressed to the Pi itself.

Q12. Why is the first rule 'ESTABLISHED,RELATED accept'?

Replies to a connection we already allowed must get back. Without it, the cloud server's answers would be dropped.

Q13. Why do you flush conntrack after installing rules?

Linux remembers open connections. Flushing them forces every existing flow to be checked against the new policy instead of continuing under the old one.

Q14. What is NAT and why is it needed?

The camera has a private address 10.42.0.17. NAT replaces it with the Pi's address on the hotspot so replies can come back. NetworkManager's 'shared' mode sets it up.

Security and the signature

Q15. Why sign MUD files?

Otherwise an attacker could give the gateway a fake file saying 'allow everything'. The signature proves the file came from the manufacturer and was not changed.

Q16. Explain RSA-PSS with SHA-256 simply.

SHA-256 makes a fixed 256-bit fingerprint of the file. The manufacturer encrypts that fingerprint with its private key; this is the signature. The gateway checks it with the public key. PSS is a modern padding method that adds randomness and is recommended over the old PKCS#1 v1.5.

Q17. Why did you move from HMAC to RSA?

HMAC uses one shared secret. Whoever can check a file can also create one, so a hacked gateway could forge MUD files. With RSA only the private-key holder can sign; the gateway only has the public key.

Q18. What if the private key is stolen?

An attacker could sign fake files. Real deployments use a certificate chain so the key can be revoked and replaced. That is part of our future work.

Q19. What happens if the MUD server is down?

The watchdog logs 'fetch failed - keeping current rules'. The device stays protected by its last good policy.

Q20. Could an attacker pretend to be the camera?

If it takes the camera's IP address, yes, it would get the camera's (strict) rules. Strong identity needs per-device keys such as 802.1X. This is a stated limitation.

Our additions

Q21. What is east-west traffic and why didn't iptables see it?

Traffic between two devices on the same network. On WiFi the radio chip can pass frames directly from laptop to camera, so they never enter Linux. Our proof: ping replies with TTL 64 and MUD counters at 0.

Q22. How did you fix it?

AP isolation stops the radio passing frames between clients. Proxy ARP with proxy_arp_pvlan makes the Pi answer ARP for every client, so their packets go through the Pi's FORWARD chain and meet the MUD rules. We also disabled ICMP redirects so the Pi doesn't tell the laptop to go direct. Result: 36/36 same-WiFi attack trials blocked, and the kernel log shows MUD_CAM_TO_DROP with TTL 127.

Q23. Why can't a firewall control MQTT topics?

The topic is inside the MQTT message. The firewall only sees TCP port 1883, so allowing the port allows every topic.

Q24. How does your MQTT extension work?

The signed MUD file has an mqtt-topic-acl section listing the device's username, publish topics and subscribe topics. mqtt_acl_manager.py verifies the file and writes Mosquitto's password file and ACL, with anonymous access off. The broker then checks every publish and subscribe.

Q25. What does '#' mean in MQTT and why block it?

It is a wildcard meaning every topic. A hacked device subscribing to '#' could read all other devices' messages.

Q26. How does the watchdog detect a change?

Every 2 s it downloads the file and compares its SHA-256 hash with the previous one. Same hash means nothing to do.

Q27. What is an atomic rule swap and why does it matter?

All new rules are loaded in one iptables-restore transaction. The old way emptied the chain first; while empty, packets fell back to FORWARD's ACCEPT policy (fail-open). Atomic means packets always see a complete policy.

Q28. Why polling instead of push?

Simple and reliable with a plain HTTP server. Detection is bounded by the interval (2 s). Push notifications are future work.

Q29. What did you learn about RFC 8520's DNS names?

The camera's STUN and relay servers are chosen at run time and never appear in DNS, so a DNS-name-only policy breaks live view. We used a hybrid policy: names for fixed services, port rules for dynamic ones.

Q30. Why was the P2P traffic blocked and did it break the camera?

The camera tried to reach the viewer's phone directly over UDP, which is not in the policy. MUD dropped it, the app fell back to the allowed TCP 1443 relay and live view kept working.

Testing and results

Q31. How did you measure enforcement accuracy?

attack_test.py tries 29 connections (5 allowed, 24 forbidden) 3 times each, first with no rules (baseline) and then with rules. Accuracy = correct decisions / all counted trials = 87/87 = 100%.

Q32. Why a baseline run?

A timeout can also mean a server is down. Only tests that succeed without MUD are counted, so every block in the enforced run is caused by MUD.

Q33. What are the canary and off-path checks?

The canary is a forbidden connection tried before and after the run; if it gets through, the rules were not active and the run is invalid. Off-path checks confirm each trial left through the MUD gateway; Windows once silently switched WiFi.

Q34. What makes up the 166 ms reload time?

About 43 ms HTTP fetch, 1.3 ms signature check, 0.9 ms parsing and 110–140 ms for the atomic install. End-to-end it is about 1.1 s because we wait for the next 2-second check.

Q35. Can someone else rebuild your setup?

Yes. It uses only off-the-shelf parts (Raspberry Pi 3B+, microSD card, USB WiFi adapter), all listed in docs/hardware_bom.csv, and every script, MUD file and test result is in the GitHub repository. This is research gap 5.6.

Q36. What happens after a power cut?

systemd starts the MUD server and the watchdog at boot. The rules were back about 10 s after the services started, with nobody touching the Pi.

Wrap-up

Q37. What is novel in your project?

Plain RFC 8520 enforcement already exists. Our contribution is closing six gaps on real hardware: east-west isolation without enterprise switches, MQTT topic control through a signed MUD extension, signed atomic hot-reload, auditable logs, and measured cost and accuracy.

Q38. What are the main limitations?

The MUD URL is configured rather than announced by the device, the signature format is simplified compared with CMS, device identity is by IP, some video rules are port-only, IPv4 only, and MQTT is without TLS.

Q39. What would you do with more time?

Automatic device identification and MUD URL discovery, certificate-based keys, a dashboard, push-based updates, IPv6, and testing more device types.

Q40. What was the hardest problem?

Getting the camera onto the Pi's access point (the AP was using WPA1 until we forced WPA2/CCMP) and discovering that same-WiFi traffic bypassed iptables. Both were solved by measuring, not guessing.